

Graphics pipeline

The **computer graphics pipeline**, or simply **graphics pipeline**, also known as the **rendering pipeline**, is a series of procedures for transforming a three-dimensional (3D) scene into a two-dimensional (2D) representation on a screen.^[1] Once a 3D model is generated, the 3D computer graphics pipeline converts the model into a visually perceivable format on the computer display.^[2]

Due to the dependence on specific software and hardware configurations and desired display attributes, a universally applicable graphics pipeline does not exist. Nevertheless, graphics application programming interfaces (APIs), such as Direct3D, OpenGL and Vulkan were developed to standardize common procedures and oversee the graphics pipeline of a given hardware accelerator. These APIs provide an abstraction layer over the underlying hardware, relieving programmers from the need to write code explicitly targeting various graphics hardware accelerators like AMD, Intel, Nvidia, and others.

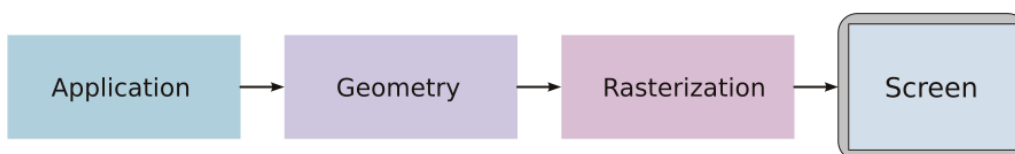
The model of the graphics pipeline is usually used in real-time rendering. Often, most of the pipeline steps are implemented in hardware, which allows for special optimizations. The term "pipeline" is used in a similar sense for the pipeline in processors: the individual steps of the pipeline often run in parallel when a given stage has enough data.

Concept

The 3D pipeline usually refers to the most common form of computer 3-dimensional rendering called 3D polygon rendering, distinct from raytracing and raycasting. In raycasting, a ray originates at the point where the camera resides, and if that ray hits a surface, the color and lighting of the point on the surface where the ray hit is calculated. In 3D polygon rendering the reverse happens – the area that is given the camera is calculated and then rays are created from every part of every surface given the camera and traced back to the camera.^[3]

Structure

A graphics pipeline can be divided into three main parts: Application, Geometry, and Rasterization.^[4]



Application

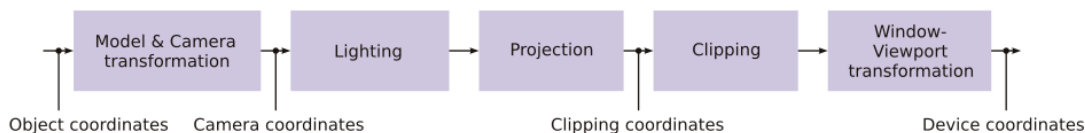
The application step is executed by the software or hardware on the main processors (CPU, GPU). During the application step, changes are made to the scene as required, for example, by user interaction using input devices or during an animation. The new scene with the modified primitives (points, lines, triangles,

etc.) is then passed on to the next step in the pipeline.

Examples of application step are collision detection, animation, morphing, and acceleration techniques using spatial subdivision schemes such as quadtrees or octrees. These are also used to reduce the amount of main memory required at a given time. The "world" of a modern computer game is usually larger than available onboard memory.

Geometry

The geometry pipeline and the vertex pipeline are responsible for most of the operations with polygons. These can be further divided into the five tasks. How these tasks are organized as actual parallel pipeline steps is implementation specific.



- Model and camera transformations
- Lighting
- Projection (Z-buffering)
- Clipping
- Viewport

World Coordinate System

The world coordinate system is the coordinate system in which the virtual world is created. The objects contained within the scene (houses, trees, cars) are often designed in their object coordinate system (also called model coordinate system or local coordinate system) for reasons of simpler modelling. To assign these objects to coordinates in the world coordinate system or global coordinate system of the entire scene, the object coordinates are transformed using translation, rotation, or scaling. This is done by multiplying the corresponding transformation matrices. In addition, several differently transformed copies can be formed from one object, for example, a forest from a tree. This is called instancing.

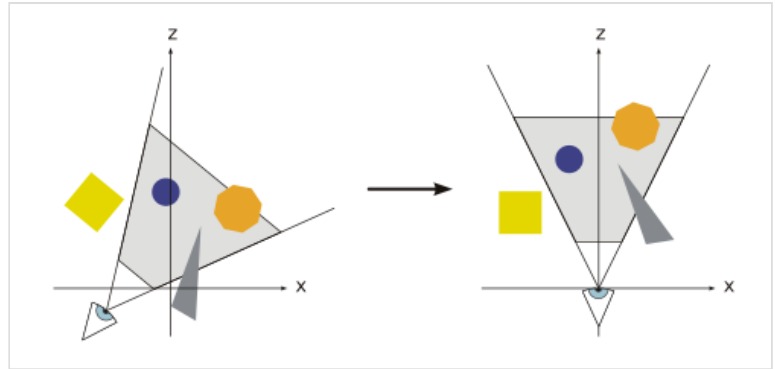
Camera Transformation

In addition to the objects, the scene also defines a virtual camera or viewer that indicates the position and direction of view relative to which the scene is rendered. The scene is transformed so that the camera is at the origin looking along the Z-axis. The resulting coordinate system is called the camera coordinate system and the transformation is called *camera transformation* or *View Transformation*.

Projection

The 3D projection step transforms the view volume into a cube with the corner point coordinates $(-1, -1, 0)$ and $(1, 1, 1)$; Occasionally other target volumes are also used. This step is called *projection*, even though it transforms a volume into another volume, since the resulting Z coordinates are not stored in the image, but are only used in Z-buffering in the later rastering step. In a perspective illustration, a central projection is used.

To limit the number of displayed objects, two additional clipping planes are used; The visual volume is therefore a truncated pyramid (frustum). The parallel or orthogonal projection is used, for example, for technical representations because it has the advantage that all parallels in the object space are also parallel in the image space, and the surfaces and volumes are the same size regardless of the distance from the viewer. Maps use, for example, an orthogonal projection (so-called orthophoto), but oblique images of a landscape cannot be used in this way – although they can technically be rendered, they seem so distorted that we cannot make any use of them. #



Left: Position and direction of the virtual viewer (camera), as defined by the user. Right: Positioning the objects after the camera transformation. The light gray area is the visible volume.

For reasons of efficiency, the camera and projection matrix are usually combined into a transformation matrix so that the camera coordinate system is omitted. The resulting matrix is usually the same for a single image, while the world matrix looks different for each object. In practice, therefore, view and projection are pre-calculated so that only the world matrix has to be adapted during the display. However, more complex transformations such as vertex blending are possible. Freely programmable geometry shaders that modify the geometry can also be executed.

In the actual rendering step, the world matrix * camera matrix * projection matrix is calculated and then finally applied to every single point. Thus, the points of all objects are transferred directly to the screen coordinate system (at least almost, the value range of the axes is still $-1..1$ for the visible range, see section "Window-Viewport-Transformation").

Lighting

Often a scene contains light sources placed at different positions to make the lighting of the objects appear more realistic. In this case, a gain factor for the texture is calculated for each vertex based on the light sources and the material properties associated with the corresponding triangle. In the later rasterization step, the vertex values of a triangle are interpolated over its surface. A general lighting (ambient light) is applied to all surfaces. It is the diffuse and thus direction-independent brightness of the scene. The sun is a directed light source, which can be assumed to be infinitely far away. The illumination effected by the sun on a surface is determined by forming the scalar product of the directional vector from the sun and the normal vector of the surface. If the value is negative, the surface is facing the sun.

Clipping

Only the primitives that are within the visual volume need to be rastered (drawn). This visual volume is defined as the inside of a frustum, a shape in the form of a pyramid with a cut-off top. Primitives that are completely outside the visual volume are discarded; This is called frustum culling. Further culling methods such as back-face culling, which reduces the number of primitives to be considered, can theoretically be executed in any step of the graphics pipeline. Primitives that are only partially inside the

cube must be clipped against the cube. The advantage of the previous projection step is that the clipping always takes place against the same cube. Only the – possibly clipped – primitives, which are within the visual volume, are forwarded to the final step.

Window-Viewport transformation

To output the image to any target area (viewport) of the screen, another transformation, the *Window-Viewport transformation*, must be applied. The resulting coordinates are the device coordinates of the output device.

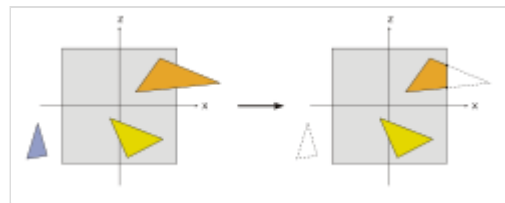
On modern hardware, most of the geometry computation steps are performed in the vertex shader. This is, in principle, freely programmable, but generally performs at least the transformation of the points and the illumination calculation. For the DirectX programming interface, the use of a custom vertex shader is necessary from version 10, while older versions still have a standard shader.

Rasterization

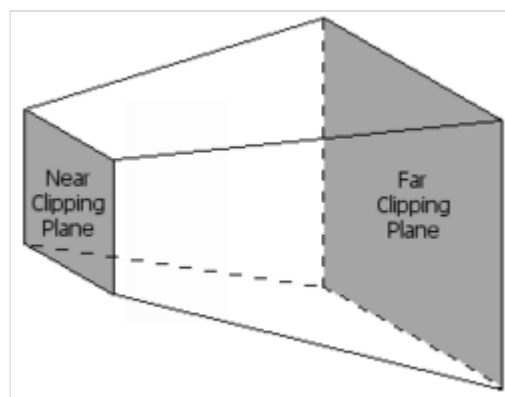
The rasterization step is the final step before the fragment shader pipeline that all primitives are rasterized with. In the rasterization step, discrete fragments are created from continuous primitives.

In this stage of the graphics pipeline, the grid points are also called fragments, for the sake of greater distinctiveness. Each fragment corresponds to one pixel in the frame buffer and this corresponds to one pixel of the screen. These can be colored (and possibly illuminated). Furthermore, it is necessary to determine the visible, closer to the observer fragment, in the case of overlapping polygons. A Z-buffer is usually used for this so-called hidden surface determination. The color of a fragment depends on the illumination, texture, and other material properties of the visible primitive and is often interpolated using the triangle vertex properties. Where available, a fragment shader (also called Pixel Shader) is run in the rastering step for each fragment of the object. If a fragment is visible, it can now be mixed with already existing color values in the image if transparency or multi-sampling is used. In this step, one or more fragments become a pixel.

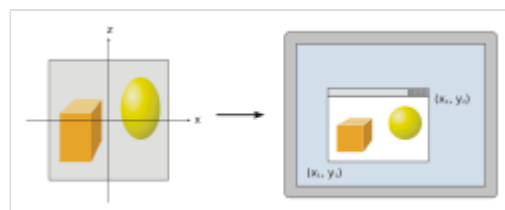
To prevent the user from seeing the gradual rasterization of the primitives, double buffering takes place. The rasterization is carried out in a special memory area. Once the image has been completely rasterized, it is copied to the visible area of the image memory.



Clipping of primitives against the cube. The blue triangle is discarded while the orange triangle is clipped, creating two new vertices.



Frustum



Window-Viewport-Transformation

Inverse

All matrices used are nonsingular and thus invertible. Since the multiplication of two nonsingular matrices creates another nonsingular matrix, the entire transformation matrix is also invertible. The inverse is required to recalculate world coordinates from screen coordinates – for example, to determine from the mouse pointer position the clicked object. However, since the screen and the mouse have only two dimensions, the third is unknown. Therefore, a ray is projected at the cursor position into the world and then the intersection of this ray with the polygons in the world is determined.

See also

- Pipeline (computing)
 - Instruction pipelining
- Hardware acceleration
- Shader

Sources

- Akenine-Möller, Tomas; Haines, Eric; Hoffman, Naty (2019) [2008]. *Real-Time Rendering*. CRC Press. ISBN 9781315362007.
- Bender, Michael; Brill, Manfred (2006). *Computergrafik: ein anwendungsorientiertes Lehrbuch*. München: Hanser. ISBN 3-446-40434-1.
- Fischer, Martin (2011-07-04). *Pixel-Fabrik. Wie Grafikchips Spielwelten auf den Schirm zaubern*. c't Magazin für Computer Technik. Heise Zeitschriften Verlag. p. 180. ISSN 0724-8679 (<https://search.worldcat.org/issn/0724-8679>).

References

1. "Graphics Pipeline" (<https://msdn.microsoft.com/en-us/library/windows/desktop/ff476882%28v=vs.85%29.aspx>). Microsoft. May 30, 2018. Retrieved 15 June 2015.
2. "Lecture: Graphics pipeline and animation" (<https://web.archive.org/web/20171207095603/http://goanna.cs.rmit.edu.au/~gl/teaching/rtr%263dgp/notes/pipeline.html>). Archived from the original (<http://goanna.cs.rmit.edu.au/~gl/teaching/rtr&3dgp/notes/pipeline.html>) on 7 December 2017. Retrieved 15 June 2015.
3. Lawrence, Jason (October 22, 2012). "3D Polygon Rendering Pipeline" (http://www.connellybarnes.com/work/class/2016/intro_gfx/lectures/13-Pipeline.pdf) (PDF). Archived (<https://web.archive.org/web/20161229102150/http://www.cs.virginia.edu/~gfx/Courses/2012/IntroGraphics/lectures/13-Pipeline.pdf>) (PDF) from the original on December 29, 2016. Retrieved 2019-09-19.
4. Akenine-Möller, Haines & Hoffman 2019, p. 13 2.1 Architecture Figure 2.2 (<https://books.google.com/books?id=Y-KEDwAAQBAJ&pg=PA13>)

External links

-

